DESENVOLVIMENTO MOBILE · MÓDULO 12 · TRILHA FLUTTER · NÍVEL INTERMEDIÁRIO

# Firebase com FlutterFire

Configuração, Authentication, Cloud Firestore, regras de segurança, Storage, Cloud Functions, mensagens push, Crashlytics e Emulator Suite

**Prof. Dr. Francisco dos Santos Viana**

São Luís – Maranhão – Brasil · 2026

# Objetivos do módulo

**Ao final deste módulo você será capaz de**

- Conectar um app Flutter ao Firebase com o FlutterFire CLI.
- Autenticar usuários por e-mail e senha e com Google.
- Modelar, ler e gravar dados no Cloud Firestore em tempo real.
- Proteger os dados com regras de segurança.
- Enviar arquivos ao Storage, chamar Cloud Functions e receber notificações push.
- Monitorar falhas com Crashlytics e desenvolver com o Emulator Suite.

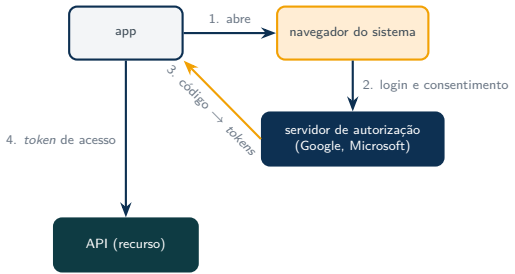
# Roteiro

## Neste módulo

- Fundamentos: identidade, tempo real e mensagens
- Visão geral
- Configuração
- Authentication
- Cloud Firestore
- Regras de segurança
- Storage, Functions e mensagens
- Monitoramento e desenvolvimento
- Fechamento

# Fundamentos: identidade, tempo real e mensagens

# Identidade federada: OAuth 2.0 e OpenID Connect




### Um pouco de história

OAuth 2.0 virou padrão em 2012 (RFC 6749); o OpenID Connect veio em 2014. Para apps nativos, a RFC 8252 (2017) recomenda o navegador do sistema e PKCE.

### OAuth 2.0 e OpenID Connect

**OAuth 2.0** delega **autorização**: o app recebe um *token* para agir em nome do usuário sem ver a senha. O **OpenID Connect** acrescenta **identidade**: um *ID token* diz quem é o usuário.

### PKCE

O app cria um segredo aleatório por login e envia só o seu *hash*; na troca do código pelos *tokens*, mostra o segredo. Quem interceptar o código não consegue usá-lo.

### Atenção

Nunca peça a senha do Google numa tela do app nem use *WebView* para o login.

# *Tokens* no app: identidade, acesso e renovação

| *Token* | Para que serve | Validade típica | Cuidados |
|---|---|---|---|
| ID *token* | informa quem é o usuário (nome, e-mail, `sub`) | 1 hora no Firebase | não serve para chamar APIs de terceiros |
| De acesso | autoriza chamadas a uma API | minutos a 1 hora | enviar só por HTTPS, no cabeçalho `Authorization` |
| De renovação | obtém novos *tokens* sem novo login | semanas a meses | guardar em armazenamento seguro; revogar no *logout* |

## Vida curta, renovação automática

*Tokens* de acesso curtos limitam o estrago se vazarem. Os SDKs do Firebase renovam o ID *token* sozinhos antes de expirar.

## Papéis dentro do *token*

O servidor pode gravar papéis no *token* (*custom claims*, como `professor: true`). As regras de segurança e as funções leem esses papéis sem consultar o banco.

# Senhas, múltiplos fatores e *passkeys*

## Fatores de autenticação

Algo que você **sabe** (senha), algo que você **tem** (celular, chave física) e algo que você **é** (digital, rosto). Combinar dois fatores diferentes é a autenticação multifator (MFA).

## O ponto fraco da senha

Senhas são reutilizadas, vazam e podem ser digitadas num site falso (*phishing*). Códigos por SMS ajudam, mas também podem ser interceptados.

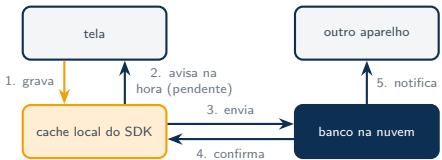
## *Passkey*

Credencial baseada em par de chaves, criada para um único site ou app. A chave privada fica no aparelho ou no gerenciador de senhas; o servidor guarda só a pública.

## Por que resiste a *phishing*

O login é a assinatura de um desafio do servidor, liberada pela biometria ou pelo PIN do aparelho. A chave só funciona no domínio para o qual foi criada, e não há segredo para vazar do servidor.

# Bancos em tempo real: escuta, cache local e compensação de latência




## Escuta (*listener*)

Em vez de perguntar repetidas vezes, o app assina uma consulta e recebe o resultado inicial e depois cada mudança, por uma conexão mantida aberta.

## Compensação de latência

A gravação aparece na tela antes de o servidor confirmar, marcada como pendente. Se o servidor recusar, o SDK desfaz a mudança.

### Escutas custam

Cada documento recebido conta como leitura. Limite as consultas (filtros, `limit`) e encerre as escutas quando a tela sair.

# Regras de segurança: a autorização mora no servidor

## O app fala direto com o banco

Em um BaaS não há servidor seu no meio: qualquer pessoa pode usar as chaves públicas do app em outro programa. Verificações feitas no app são só experiência de uso; a proteção real são as **regras**, avaliadas no servidor a cada requisição.

**firestore.rules (trecho)**

```
match /reservas/{id} {
  allow read: if request.auth.uid == resource.data.uid;
  allow create: if request.auth.uid
                   == request.resource.data.uid;
}
```

## Menor privilégio

Comece negando tudo e libere só o necessário, para quem precisa. Valide também o formato: campos obrigatórios, tipos e tamanhos.

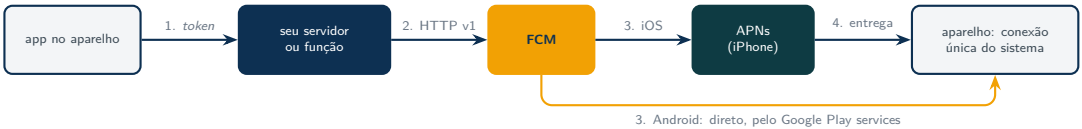
### Regras não são filtros

A regra não esconde documentos de uma consulta: se a consulta **puder** trazer algo proibido, ela inteira falha. A consulta precisa ter os mesmos filtros da regra.

### Dica

Teste as regras no emulador, com casos permitidos e negados, como qualquer outro código.

# Notificações *push* por dentro




## *Token* de registro

Endereço do app naquele aparelho. Muda em reinstalações e após limpar dados: mantenha-o atualizado no servidor.

## Dois tipos de mensagem

**Notificação**: o sistema exibe sozinho. **Dados**: o código do app decide o que fazer.

## Entrega não garantida

O usuário pode negar a permissão (Android 13+ e iOS), e o sistema adia mensagens para poupar bateria.

# Funções acionadas por eventos




**Dica**

Outros gatilhos: requisição HTTP, conta criada, arquivo enviado e horário agendado.

**Arquitetura orientada a eventos**

Componentes reagem a fatos (“uma leitura foi gravada”) em vez de chamarem uns aos outros. Quem grava não precisa saber quem reage.

**Pelo menos uma vez**

O provedor garante executar a função ao menos uma vez por evento, e às vezes executa duas. Escreva funções **idempotentes**: processar o mesmo evento de novo não pode duplicar o *push* nem o resumo.

**Onde guardar segredos**

Chaves de serviços pagos ficam no servidor e nunca no app. O App Check (Play Integrity, App Attest) ajuda a recusar chamadas que não vêm do app legítimo.

# Visão geral

# FlutterFire: os plugins do Firebase

| Pacote | Serviço |
|---|---|
| `firebase_core` | inicialização (obrigatório) |
| `firebase_auth` | autenticação |
| `cloud_firestore` | banco NoSQL em tempo real |
| `firebase_storage` | arquivos (fotos, PDFs) |
| `cloud_functions` | código no servidor |
| `firebase_messaging` | notificações push (FCM) |
| `firebase_crashlytics` | relatórios de falhas |
| `firebase_analytics` | eventos de uso |
| `firebase_remote_config` | configuração remota |
| `firebase_app_check` | só o seu app acessa o backend |
| `firebase_ai` | modelos Gemini (Módulo 20) |

**Planos**

**Spark**: gratuito, sem cartão; ao atingir a cota, o serviço para.
**Blaze**: pague pelo uso, mantendo as cotas gratuitas.

**Cloud Storage exige Blaze**

Desde **03/02/2026**, o Storage só funciona no plano Blaze. Configure um **alerta de orçamento**.

**Dica**

O Módulo 16 compara o Firebase a outros BaaS.

# Configuração

# FlutterFire CLI

```
$ npm install -g firebase-tools
$ firebase login
$ dart pub global activate flutterfire_cli
$ flutterfire configure
$ flutter pub add firebase_core firebase_auth \
    cloud_firestore
```

## O que o `configure` faz

Cria ou escolhe o projeto no console, registra os apps Android, iOS, web e macOS e gera `lib/firebase_options.dart` com as chaves de cada plataforma.

**main.dart**

```dart
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  await Firebase.initializeApp(
    options: DefaultFirebaseOptions.currentPlatform,
  );
  runApp(const LabMonitorApp());
}
```

### Dica

Rode `flutterfire configure` de novo sempre que adicionar uma plataforma ou um plugin que exija configuração nativa.

# Authentication

# E-mail e senha no repositório

data/repositories/auth_repository.dart

```dart
class AuthRepository extends ChangeNotifier {
  AuthRepository() {
    _auth.authStateChanges().listen((_) => notifyListeners());
  }
  final _auth = FirebaseAuth.instance;
  User? get usuario => _auth.currentUser;
  bool get logado => usuario != null;

  Future<Result<void>> entrar(String email, String senha) async {
    try {
      await _auth.signInWithEmailAndPassword(
          email: email, password: senha);
      return const Ok(null);
    } on FirebaseAuthException catch (e) {
      return Falha(Exception(switch (e.code) {
        'invalid-credential' => 'E-mail ou senha incorretos.',
        _ => 'Falha no login (${e.code}).',
      }));
    }
  }
}
```

## Estado da sessão

`authStateChanges()` emite o usuário ao entrar, ao sair e ao abrir o app (a sessão fica salva). O repositório vira a fonte da verdade do login.

## Outras operações

`createUserWithEmailAndPassword`, `sendPasswordResetEmail` e `signOut()`.

## Atenção

Com a proteção contra enumeração de e-mails, usuário ou senha errados viram `invalid-credential`.

# Login com Google e proteção de rotas

**Google (google_sign_in 7)**

```dart
Future<UserCredential> entrarComGoogle() async {
  if (kIsWeb) {
    return FirebaseAuth.instance
        .signInWithPopup(GoogleAuthProvider());
  }
  await GoogleSignIn.instance.initialize();
  final conta = await GoogleSignIn.instance.authenticate();
  final cred = GoogleAuthProvider.credential(
      idToken: conta.authentication.idToken);
  return FirebaseAuth.instance.signInWithCredential(cred);
}
```

**Atenção**

Android: cadastre as impressões SHA-1 e SHA-256 da chave no console. iOS: adicione o esquema de URL do cliente no `Info.plist`.

**go_router (Módulo 07)**

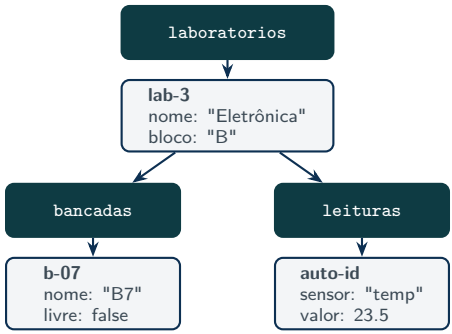
```dart
GoRouter(
  refreshListenable: auth,
  redirect: (ctx, state) {
    final noLogin =
        state.matchedLocation == '/login';
    if (!auth.logado) {
      return noLogin ? null : '/login';
    }
    return noLogin ? '/painel' : null;
  },
  routes: [ ... ],
);
```

**App Store**

Quem oferece login social de terceiros precisa oferecer também uma opção equivalente focada em privacidade, como **Sign in with Apple**.

# Cloud Firestore

# Modelo de dados: coleções e documentos




**Estrutura**

**Coleções** guardam **documentos**; documentos guardam campos e podem ter **subcoleções**. Cada documento tem até 1 MiB.

**Modele pelas telas**

Sem `JOIN`: guarde junto o que é lido junto. Duplicar um campo (como o nome do laboratório na reserva) é normal e intencional.

**Atenção**

Cobrança por leitura de documento: uma lista de 1 000 itens custa 1 000 leituras. Use `limit` e paginação.

# Lendo e gravando com `withConverter`

**data/repositories/bancada_repository_firestore.dart**

```dart
class BancadaRepositoryFirestore {
  BancadaRepositoryFirestore(this.labId);
  final String labId;
  late final _col = FirebaseFirestore.instance
      .collection('laboratorios/$labId/bancadas')
      .withConverter<Bancada>(
        fromFirestore: (s, _) => Bancada.deMapa(s.id, s.data()!),
        toFirestore: (b, _) => b.paraMapa(),
      );

  Stream<List<Bancada>> observarLivres() => _col
      .where('livre', isEqualTo: true)
      .orderBy('nome').limit(50)
      .snapshots()
      .map((q) => q.docs.map((d) => d.data()).toList());

  Future<void> criar(Bancada b) => _col.add(b);
  Future<void> renomear(String id, String nome) =>
      _col.doc(id).update({'nome': nome});
  // remover: _col.doc(id).delete()
}
```

## `snapshots()`

`Stream` que emite a lista sempre que um documento muda, em qualquer aparelho.

## Offline

Android e iOS mantêm cache local: lê sem rede e envia as gravações quando ela volta.

**Dica**

Filtro e ordenação em campos diferentes pedem um **índice composto**.

# Transações, lotes e paginação

**Reserva sem conflito**

```
Future<void> reservar(String id, String uid) =>
    FirebaseFirestore.instance.runTransaction((tx) async {
      final ref = _col.doc(id);
      final snap = await tx.get(ref);
      if (snap.data()?.livre != true) {
        throw StateError('Bancada já reservada');
      }
      tx.update(ref, {
        'livre': false,
        'reservadaPor': uid,
        'reservadaEm': FieldValue.serverTimestamp(),
      });
    });
```

**Próxima página**

```
final pagina = await _col.orderBy('nome')
    .startAfterDocument(ultimo).limit(20).get();
```

**Transação**

Lê e grava de forma atômica. Se outro aparelho alterar a bancada no meio, o Firestore repete a função. Dois alunos nunca reservam a mesma bancada.

**Lote (`batch`)**

Várias gravações tudo-ou-nada, sem leitura: `batch()`, `set`, `update`, `delete` e `commit()`.

**Dica**

`FieldValue.serverTimestamp()` usa o relógio do servidor, não o do celular.

# Regras de segurança

# Regras do Firestore

**firestore.rules**

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    function logado() { return request.auth != null; }

    match /laboratorios/{lab}/bancadas/{id} {
      allow read: if logado();
      allow update: if logado() && request.resource.data
        .diff(resource.data).affectedKeys()
        .hasOnly(['livre', 'reservadaPor', 'reservadaEm']);
      allow create, delete:
        if request.auth.token.admin == true;
    }
    match /laboratorios/{lab}/leituras/{id} {
      allow read: if logado();
      allow create: if logado()
        && request.resource.data.valor is number;
    }
  }
}
```

**O erro mais grave**

Publicar com `allow read, write: if true;` (“modo de teste”). Qualquer pessoa lê e apaga todo o banco.

**Como pensar**

O app **não é confiável**: qualquer um pode chamar a API com o seu `firebase_options`. As regras, no servidor, são a barreira real.

**Dica**

`admin` é uma *custom claim* definida por uma Cloud Function ou pelo Admin SDK.

# Storage, Functions e mensagens

# Storage e Cloud Functions

**Foto do equipamento**

```
Future<String> enviarFoto(String id, File foto) async {
  final ref = FirebaseStorage.instance
      .ref('equipamentos/$id.jpg');
  final tarefa = ref.putFile(foto,
      SettableMetadata(contentType: 'image/jpeg'));
  tarefa.snapshotEvents.listen((s) => progresso.value =
      s.bytesTransferred / s.totalBytes);
  await tarefa;
  return ref.getDownloadURL();
}
```

**Dica**

O Storage tem regras próprias (`storage.rules`): limite tamanho e tipo do arquivo.

**Chamando uma função**

```
final gerar = FirebaseFunctions.instance
    .httpsCallable('gerarRelatorio');
final r = await gerar.call({'lab': 'lab-3',
    'mes': '2026-10'});
final url = r.data['url'] as String;
```

**Cloud Functions**

Código em Node.js ou Python que roda no servidor: lógica que não pode ficar no app, envio de push, integração com outros serviços. Exige plano Blaze.

# Notificações push com FCM

**Configurando o Firebase Cloud Messaging**

```
@pragma('vm:entry-point')
Future<void> _emSegundoPlano(RemoteMessage m) async {
  await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform);
  // gravar no banco local, atualizar contador...
}

Future<void> configurarPush() async {
  FirebaseMessaging.onBackgroundMessage(_emSegundoPlano);
  final fcm = FirebaseMessaging.instance;
  await fcm.requestPermission();        // iOS e Android 13+
  final token = await fcm.getToken();   // enviar ao backend
  fcm.onTokenRefresh.listen(salvarToken);
  FirebaseMessaging.onMessage.listen(mostrarLocal);
  FirebaseMessaging.onMessageOpenedApp.listen(
      (m) => router.go('/alertas/${m.data['id']}'));
}
```

## Três situações

**Primeiro plano**: `onMessage`; você exibe.
**Fundo ou fechado**: o sistema exibe; o *handler* roda isolado.
**Toque**: `onMessageOpenedApp` abre a tela.

**Atenção**

iOS exige a chave APNs no console e a capacidade *Push Notifications* no Xcode.

# Monitoramento e desenvolvimento

# Crashlytics, Analytics e Remote Config

**main.dart: capturando falhas**

```dart
FlutterError.onError =
    FirebaseCrashlytics.instance.recordFlutterFatalError;

PlatformDispatcher.instance.onError = (erro, pilha) {
  FirebaseCrashlytics.instance
      .recordError(erro, pilha, fatal: true);
  return true;
};
```

**Evento e configuração remota**

```dart
await FirebaseAnalytics.instance.logEvent(
    name: 'reserva_criada', parameters: {'lab': 'lab-3'});

final rc = FirebaseRemoteConfig.instance;
await rc.fetchAndActivate();
final intervalo = rc.getInt('intervalo_leitura_s');
```

**Para quê**

**Crashlytics**: falhas com pilha e dispositivo.
**Analytics**: o que os usuários fazem.
**Remote Config**: mudar valores e ligar recursos sem nova versão na loja.
**App Check**: recusa chamadas que não vêm do seu app.

**LGPD**

Informe na política de privacidade os dados coletados e não envie dados pessoais em eventos.

# Emulator Suite

```
$ firebase init emulators
$ firebase emulators:start
  Auth      localhost:9099
  Firestore localhost:8080
  Storage   localhost:9199
  UI        localhost:4000
```

**Apontando o app para o emulador**

```dart
if (kDebugMode) {
  final host = defaultTargetPlatform ==
      TargetPlatform.android ? '10.0.2.2' : 'localhost';
  await FirebaseAuth.instance.useAuthEmulator(host,
      9099);
  FirebaseFirestore.instance
      .useFirestoreEmulator(host, 8080);
}
```

**Vantagens**

Sem custo, sem tocar nos dados reais; testa as regras antes de publicar e serve aos testes de integração (Módulo 14).

**Dica**

No emulador Android, `10.0.2.2` é o `localhost` do computador.

**Atenção**

Não use o Firebase direto nos widgets: encapsule em repositórios (Módulo 08).

# Fechamento

# Resumo

## Resumo

- `flutterfire configure` gera as opções; `Firebase.initializeApp` inicia tudo.
- Authentication com e-mail, Google e Apple; `authStateChanges` protege as rotas.
- Firestore: coleções, documentos, `snapshots()` em tempo real, transações e cache offline.
- Regras de segurança são a única proteção real dos dados.
- Storage (Blaze), Functions, FCM, Crashlytics e Remote Config completam o backend.

# Glossário (1/2)

| Termo | Significado |
|---|---|
| **BaaS** | *backend* pronto oferecido como serviço: contas, banco, arquivos |
| **FlutterFire** | plugins oficiais do Firebase para Flutter |
| **OAuth 2.0** | delegação de autorização por *tokens*, sem expor a senha |
| **OpenID Connect** | camada de identidade sobre o OAuth (ID *token*) |
| **PKCE** | segredo por login que impede o uso de um código interceptado |
| **ID *token*** | JWT que diz quem é o usuário; vale 1 hora no Firebase |
| ***Custom claims*** | papéis gravados no *token* pelo servidor |
| ***Passkey*** | credencial de par de chaves, resistente a *phishing* |
| `flutterfire configure` | gera o `firebase_options.dart` de cada plataforma |

# Glossário (2/2)

| Termo | Significado |
|---|---|
| **Cloud Firestore** | banco de documentos com sincronização em tempo real |
| `snapshots()` | fluxo com o resultado de uma consulta e cada mudança |
| **Compensação de latência** | a gravação aparece antes da confirmação do servidor |
| **Regras de segurança** | autorização declarativa avaliada no servidor a cada requisição |
| **FCM** | serviço de mensagens *push* do Firebase |
| ***Token* de registro** | endereço do app em um aparelho, para receber *push* |
| **Idempotência** | processar o mesmo evento duas vezes dá o mesmo resultado |
| **Emulator Suite** | emuladores locais de Auth, Firestore, Functions e Storage |

# Questões de revisão

1. Diferencie OAuth 2.0 de OpenID Connect. Qual deles o “Entrar com Google” usa?
2. Por que apps nativos devem fazer login no navegador do sistema, com PKCE?
3. Para que servem o ID *token*, o *token* de acesso e o *token* de renovação?
4. Por que *passkeys* resistem a *phishing* e senhas não?
5. O que é compensação de latência? Como a tela sabe que um documento ainda está pendente?
6. Por que as regras do Firestore são a verdadeira proteção dos dados? O que significa “regras não são filtros”?
7. Descreva o caminho de um alerta *push* da função até um iPhone e até um Android.
8. Por que uma função acionada por evento precisa ser idempotente?

# Exercícios

**Exercício 1 — Autenticação**

Implemente cadastro, login, recuperação de senha e saída, com mensagens de erro em português.

**Exercício 2 — Regras**

Escreva regras em que só o autor pode cancelar a própria reserva e teste-as no Emulator Suite.

**Exercício 3 — Push**

Envie uma notificação pelo console e faça o toque abrir a tela de alertas.

**LabMonitor — Etapa 12**

Login com e-mail e Google, bancadas e reservas no Firestore com transação, fotos no Storage e Crashlytics ativo.

# Para aprofundar

- FlutterFire: `https://firebase.flutter.dev`
- Firebase para Flutter: `https://firebase.google.com/docs/flutter/setup`
- Authentication: `https://firebase.google.com/docs/auth/flutter/start`
- Cloud Firestore: `https://firebase.google.com/docs/firestore`
- Regras de segurança: `https://firebase.google.com/docs/rules`
- Emulator Suite: `https://firebase.google.com/docs/emulator-suite`

# Respostas das questões de revisão

1. O OAuth 2.0 delega autorização (*token* de acesso); o OpenID Connect acrescenta identidade (ID *token*). O login com Google usa OpenID Connect: o app recebe um ID *token* e o troca por uma credencial do Firebase.
2. O navegador do sistema mantém a senha longe do app e reaproveita a sessão já aberta; o PKCE impede que um código interceptado seja trocado por *tokens*.
3. ID *token*: quem é o usuário. Acesso: autoriza chamadas a uma API. Renovação: obtém novos *tokens* sem novo login.
4. A *passkey* só funciona no domínio para o qual foi criada e assina um desafio sem enviar segredo. A senha pode ser digitada num site falso e reutilizada.
5. A gravação aparece na hora, antes da confirmação do servidor; se ele recusar, o SDK desfaz a mudança. Enquanto isso, `metadata.hasPendingWrites` é verdadeiro.
6. O app fala direto com o banco e qualquer um pode usar as chaves públicas: só as regras, avaliadas no servidor, barram o acesso. Uma consulta que **possa** trazer documentos proibidos falha inteira; ela precisa ter os mesmos filtros da regra.
7. A função chama o FCM pela API HTTP v1 com o *token* do aparelho. No Android, o FCM entrega pelo Google Play services; no iPhone, repassa ao APNs, que entrega pela conexão do sistema.
8. O provedor garante pelo menos uma execução por evento e às vezes repete; sem idempotência, o mesmo alerta seria enviado duas vezes.

# Fim do Módulo 12

**Próximo:** Módulo 13 — Recursos do dispositivo: câmera, GPS, sensores, Bluetooth, NFC, notificações e IoT.